FLOODTASK

LEARNING GUIDE

FloodTask: the project from A to Z

A learning guide in Simplified Technical English (ASD-STE100): basics, data, models, reinforcement learning, results

Forecast-aware reinforcement learning for multi-pass satellite imaging of evolving floods

Study region: Assam, India (Brahmaputra valley)

Edition: 25 October 2026 | numbers from the saved project results

Contents

How to use this guide . . . 4

Level 0. AI and machine learning basics . . . 5

0.1 What is artificial intelligence? . . . 5

0.2 The three types of machine learning . . . 5

0.3 Neural networks in short . . . 5

Level 1. Reinforcement learning basics . . . 7

1.1 The agent and the environment . . . 7

1.2 Return and discount . . . 7

1.3 Policy and value . . . 8

1.4 MDP and POMDP . . . 8

1.5 Types of RL algorithms . . . 9

Level 2. The problem . . . 10

2.1 Floods in Assam . . . 10

2.2 Why radar satellites? . . . 10

2.3 The tasking problem . . . 10

2.4 Why reinforcement learning? . . . 12

Level 3. The data . . . 13

3.1 The data sources . . . 13

3.2 The event split . . . 14

3.3 Terrain: HAND . . . 14

Level 4. The flood simulator and the sensor model . . . 16

4.1 Why a simulator? . . . 16

4.2 How the simulator works . . . 16

4.3 The sensor model . . . 17

Level 5. The RL environment . . . 18

5.1 Step, pass and episode . . . 18

5.2 The observation . . . 18

5.3 The action and the mask . . . 19

5.4 The shared map estimator . . . 19

5.5 The reward . . . 19

Level 6. The neural networks . . . 21

6.1 U-Net for flood segmentation . . . 21

6.2 The actor-critic network . . . 21

6.3 From scores to an action . . . 22

Level 7. The learning algorithm: PPO . . . 24
7.1 Policy gradient: the main idea . . . 24
7.2 The advantage with GAE . . . 24
7.3 The clipped objective . . . 24
7.4 The full loss . . . 25
7.5 One PPO iteration . . . 25
Level 8. Baselines and training . . . 27
8.1 Baselines (hand-made policies) . . . 27
8.2 The five PPO variants . . . 27
8.3 Why five seeds? . . . 28
8.4 The training runs . . . 28
Level 9. Evaluation and results . . . 29
9.1 The main metric: IoU . . . 29
9.2 How the comparison is fair . . . 29
9.3 Main result . . . 29
9.4 Robustness . . . 30
9.5 Requirements . . . 31
Level 10. Real events and the flood outlook . . . 32
10.1 Real-event replay . . . 32
10.2 The flood outlook . . . 32
Level 11. The code and how to run it . . . 34
11.1 Folder map . . . 34
11.2 The pipeline . . . 34
11.3 The viewer app . . . 35
11.4 An episode picture . . . 36
Level 12. Limits and honest statements . . . 37
Level 13. Glossary and viva questions . . . 38
13.1 Glossary . . . 38
13.2 Likely viva questions . . . 39

How to use this guide

This guide explains the FloodTask project from the start to the end. It does not think that you know machine learning. Each level adds one layer of knowledge. Each level has text and one or more diagrams.

All numbers in this guide come from the result files of the project. The guide does not estimate or invent a result. When a part is simulated or synthetic, the guide tells you.

The writing rules (ASD-STE100)

The text follows the ASD-STE100 Simplified Technical English rules. These rules make technical text easy to read:

- One sentence gives one idea.
- A sentence for a procedure has 20 words or fewer. A sentence for a description has 25 words or fewer.
- The text uses the active voice. Example: "The agent selects a tile", not "A tile is selected".
- Procedures are numbered steps. Each step starts with a verb (for example, "Run", "Open").
- Each technical term is defined when it first appears. The Glossary (Level 13) repeats all definitions.
- Warnings and cautions are in coloured boxes.

The levels

Level	Topic	You learn
0	AI and machine learning basics	What AI, ML and the three types of learning are
1	Reinforcement learning (RL) basics	Agent, state, action, reward, return, policy, value, MDP, POMDP
2	The problem	Why flood maps need satellite tasking, and why RL fits
3	Data	The five data sources, the event split and the terrain (HAND)
4	Flood simulator and sensor model	How the project makes flood movies and fake satellite images
5	The RL environment	Observation, action, mask, reward, episode
6	The neural networks	U-Net (for images) and the actor-critic (for decisions)
7	The learning algorithm: PPO	Policy gradient, advantage, GAE, clipping, loss
8	Baselines and training	Hand-made policies, 25 training runs, model selection
9	Evaluation and results	Metrics, statistics and every main result
10	Real events and the flood outlook	Assam 2016 and 2022 replays, 4-24 h outlook
11	The code and how to run it	Folders, modules, commands, the viewer app
12	Limits and honest statements	What the project does not show
13	Glossary and viva questions	Short definitions and likely questions with answers

ONE-SENTENCE SUMMARY

The project trains a reinforcement learning agent to select which 16 of 256 map tiles a radar satellite images at each pass. The goal is a flood map of Assam that stays correct and up to date.

Level 0. AI and machine learning basics

0.1 What is artificial intelligence?

Artificial intelligence (AI) is a computer program that does a task which usually needs human thinking. Examples are to read text, to find objects in a photo, or to make a plan.

Machine learning (ML) is one part of AI. An ML program does not get hand-written rules. It finds the rules from data. We call this process **training**. The result of training is a **model**.

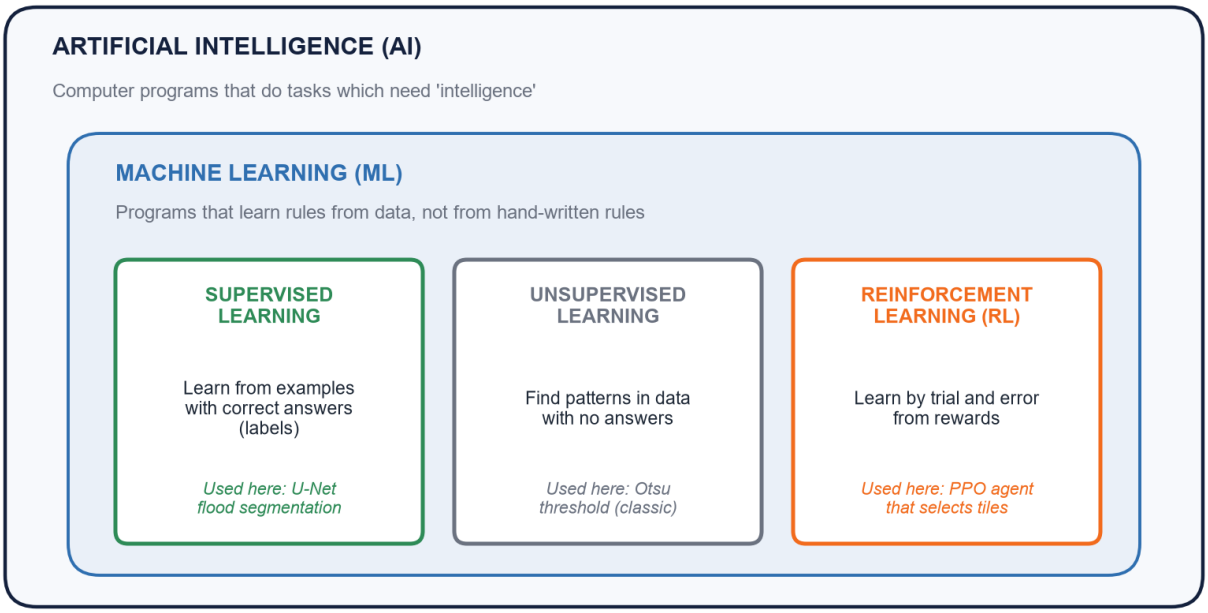


Figure 0.1 - AI contains ML. ML has three main types. This project uses two of them.

0.2 The three types of machine learning

Type	Data that it gets	What it learns	Use in this project
Supervised learning	Inputs and the correct answer for each input (labels)	A function from input to answer	The U-Net learns to find water in radar images
Unsupervised learning	Inputs only, no answers	Groups and patterns	The classic Otsu threshold (a reference, not trained)
Reinforcement learning	No answers. Only a reward after each action	A strategy of actions	The PPO agent learns which tiles to image

0.3 Neural networks in short

A **neural network** is a model made of layers. Each layer multiplies its inputs by numbers and adds numbers. Then it applies a simple non-linear function, for example ReLU (ReLU(x) = max(0, x)). The numbers are the **parameters** (or weights).

Training changes the parameters a little at a time. A **loss function** measures how wrong the model is. The **gradient** tells the direction that makes the loss smaller. The optimizer (this project uses Adam) moves the parameters in that direction. One small move is one **update step**.

- A **convolutional neural network (CNN)** uses small filters that move over an image. It finds local shapes, for example a river edge.
- A **recurrent neural network**, for example an **LSTM** (long short-term memory), keeps a memory vector from one step to the next step.
- A **GPU** (graphics processing unit) does many multiplications at the same time. The project trains on an NVIDIA RTX 4050 laptop GPU (6 GB).

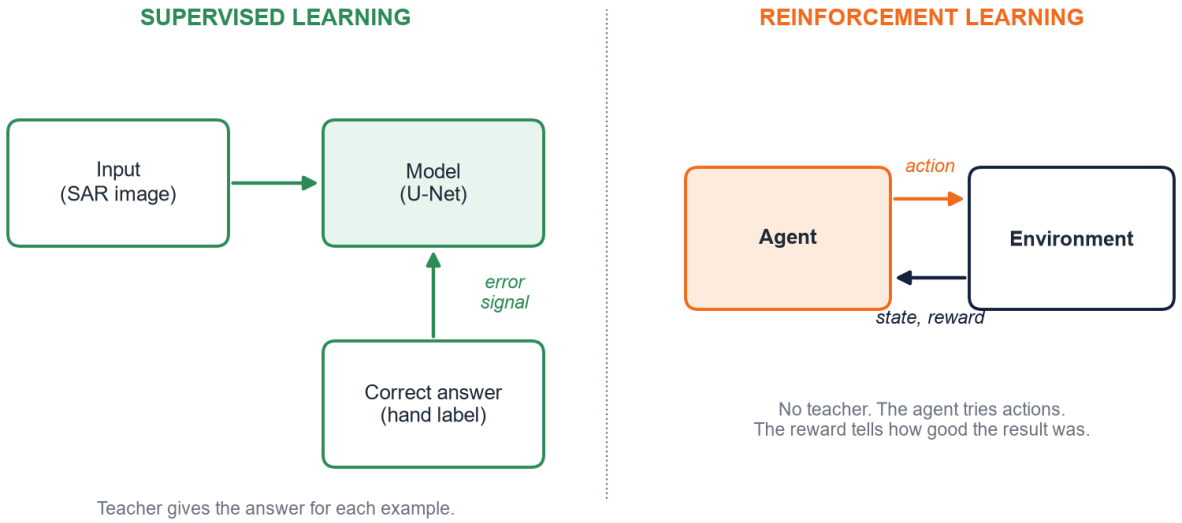


Figure 0.2 - Supervised learning has a teacher. Reinforcement learning has only a reward.

REMEMBER

Supervised learning answers the question "what is in this image?".

Reinforcement learning answers the question "what do I do next?".

Level 1. Reinforcement learning basics

1.1 The agent and the environment

Reinforcement learning (RL) has two parts. The **agent** makes decisions. The **environment** is everything outside the agent. In this project, the agent is a neural network. The environment is a flood simulator with a satellite.

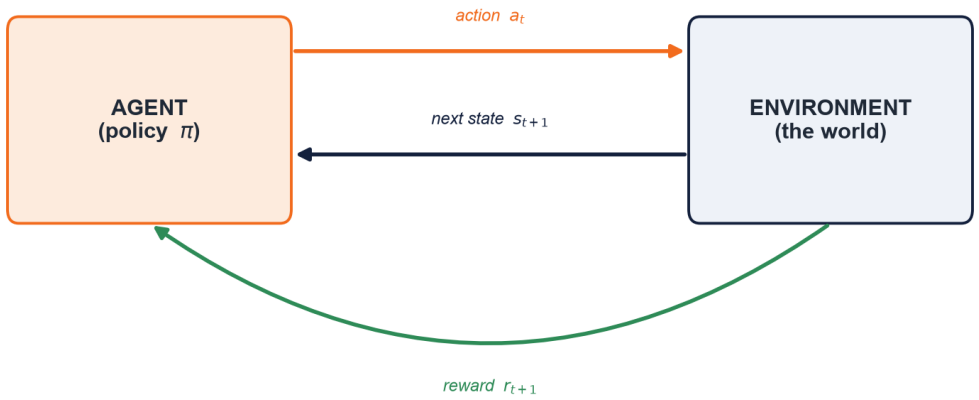


Figure 1.1 - The RL loop. It repeats at each time step.

At each time step t, these five things occur:

1. The agent gets the **state** s_t (a description of the world now).
2. The agent selects an **action** a_t.
3. The environment changes because of the action and because of time.
4. The environment gives a **reward** r_{t+1} (one number) and the next state s_{t+1}.
5. The loop starts again. It stops at the end of the **episode**.

An **episode** is one full run from the start to the end. In this project, one episode is one flood event in one map area.

1.2 Return and discount

The agent does not try to get the largest reward now. It tries to get the largest **sum of rewards** in the future. This sum is the **return** G.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- G_t is the return from time t.
- r is the reward at each step.
- γ (gamma) is the **discount factor**, between 0 and 1. A reward k steps in the future counts γ^k times.
- This project uses γ = 0.9. A reward 10 steps later counts only 0.9^{10} = 0.35 times.

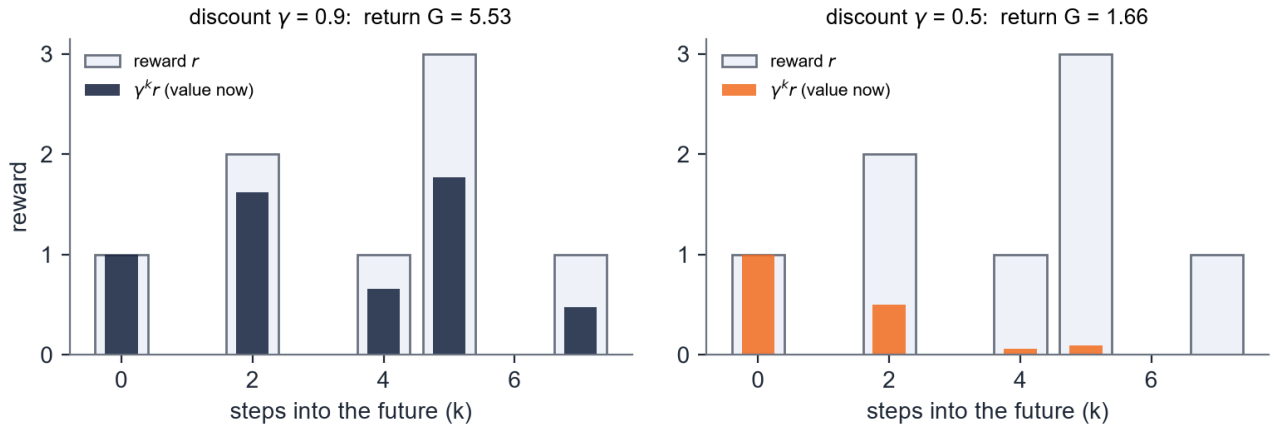


Figure 1.2 - The same rewards with two discount values. A small γ makes the agent short-sighted.

1.3 Policy and value

The **policy** π is the strategy of the agent. It gives the probability of each action in each state: π(a | s). In this project, the policy gives one probability for each of the 256 tiles.

The **value function** V(s) tells how good a state is. It is the expected return when the agent starts in s and follows π.

$$V^{\pi}(s) = \mathbb{E}_{\pi}[\, G_t \mid s_t = s \,]$$

- E means "expected value" (the average over many runs).

The **Bellman equation** connects the value now with the value at the next step:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[\, r_{t+1} + \gamma V^{\pi}(s_{t+1}) \mid s_t = s \,]$$

- Value now = reward now + discounted value of the next state.
- The critic network (Level 6) learns V by this relation.

The **Q-value** Q(s, a) is the expected return when the agent does action a first. The **advantage** A(s, a) tells how much better action a is than the average action:

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t \mid s_t = s,\ a_t = a], \qquad A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

- A > 0: the action is better than usual. The agent must do it more often.
- A < 0: the action is worse than usual. The agent must do it less often.

1.4 MDP and POMDP

A **Markov decision process (MDP)** is the mathematical frame of RL. In an MDP, the agent sees the full state. The next state depends only on the current state and the action (the Markov property).

In this project, the agent cannot see the true flood. It sees only old and noisy satellite images. This is a **partially observable MDP (POMDP)**. The agent gets an **observation** o, not the state s.

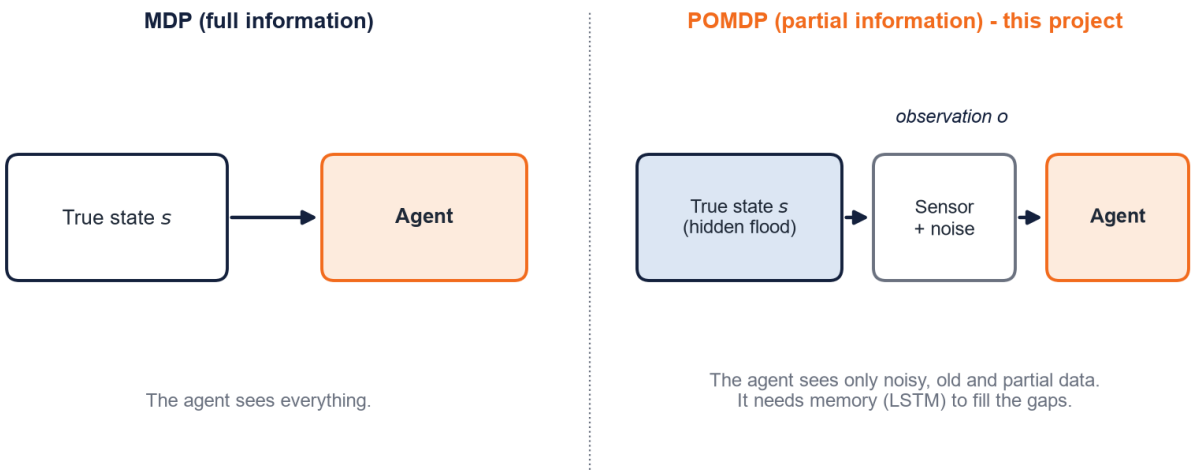


Figure 1.3 - MDP compared with POMDP. FloodTask is a POMDP.

WHY MEMORY?

In a POMDP, one observation does not show everything. An agent with memory (an LSTM) can remember what it saw before. The project tests if this memory helps (Level 9).

1.5 Types of RL algorithms

Family	Main idea	Examples
Value-based	Learn Q(s, a). Select the action with the largest Q.	Q-learning, DQN
Policy-gradient	Learn π directly. Increase the probability of good actions.	REINFORCE
Actor-critic	An actor learns π. A critic learns V and gives the advantage.	A2C, **PPO (used here)**

The project uses **PPO** (proximal policy optimisation). PPO is stable, simple and good for large discrete action sets with masks. Level 7 explains PPO step by step.

Level 2. The problem

2.1 Floods in Assam

Assam is a state in north-east India. The Brahmaputra river flows through the Assam valley. Monsoon rain floods large areas of this valley almost every year. Rescue teams and relief agencies need a flood map. The map must show where the water is now.

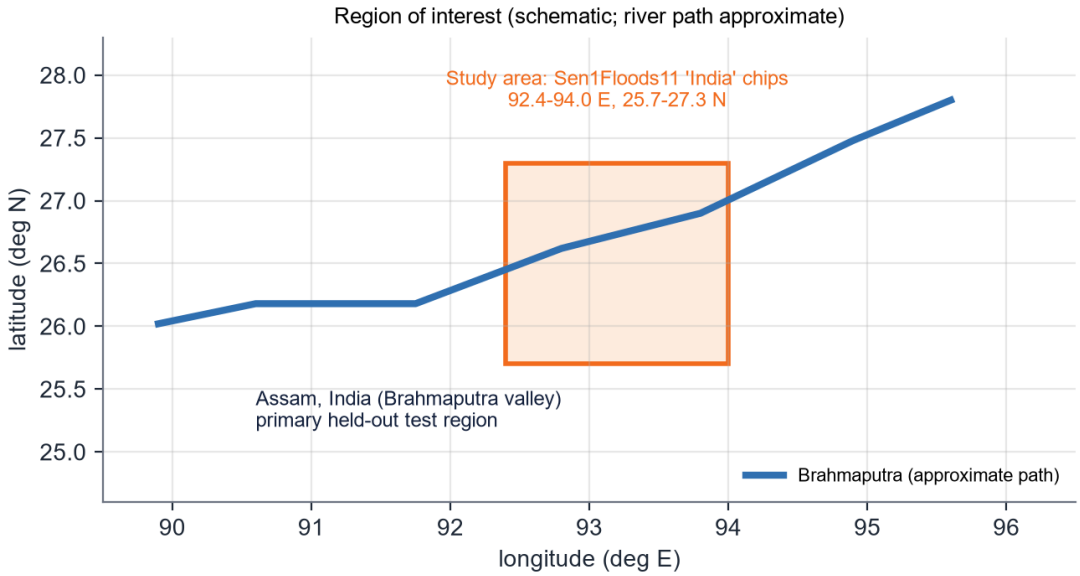


Figure 2.1 - The study area. The test scenes come from the Sen1Floods11 'India' chips in Assam.

2.2 Why radar satellites?

Clouds cover Assam during the monsoon. Optical satellites cannot see through clouds. **Synthetic aperture radar (SAR)** sends microwaves and measures the echo. Microwaves go through clouds. Calm water reflects the microwaves away from the satellite, so water looks dark in a SAR image. Sentinel-1 is a free SAR satellite of the European Space Agency.

2.3 The tasking problem

A modern agile SAR satellite can point its camera. It can image only a small number of areas at each pass. The choice of areas is called **tasking**. The flood moves while the satellite passes again and again.

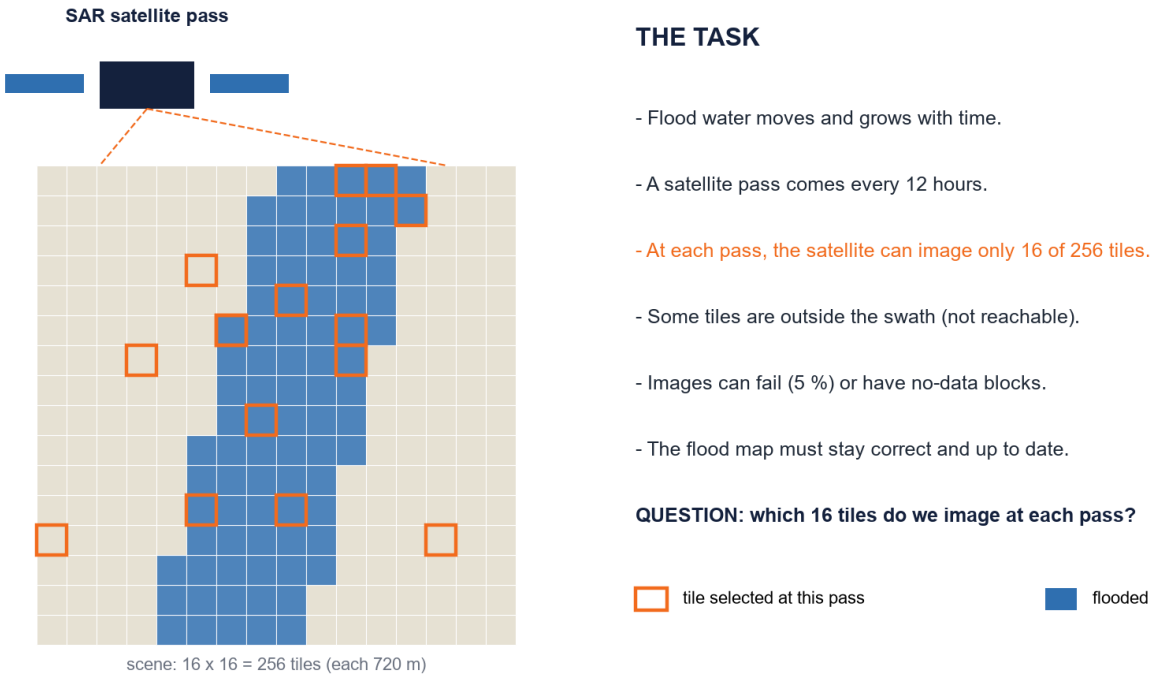


Figure 2.2 - The task. Select 16 of 256 tiles at each pass, so the map stays correct.

The project uses these fixed numbers (file configs/env.yaml):

Item	Value	Meaning
Scene	192 x 192 cells of 60 m	One map area of 11.5 km x 11.5 km
Tile	12 x 12 cells (720 m)	The unit that the satellite can image; 256 tiles per scene
Passes	12, every 12 h	Six days of observation after a 48 h spin-up
Budget	16 tiles per pass	Only 6.25 % of the scene per pass
Swath	8-16 tile rows or columns; full scene with probability 0.25	Some tiles are not reachable at a pass
Failures	5 % of acquisitions	The image is lost
Evaluation time	pass time + 3 h	The map is scored after processing time

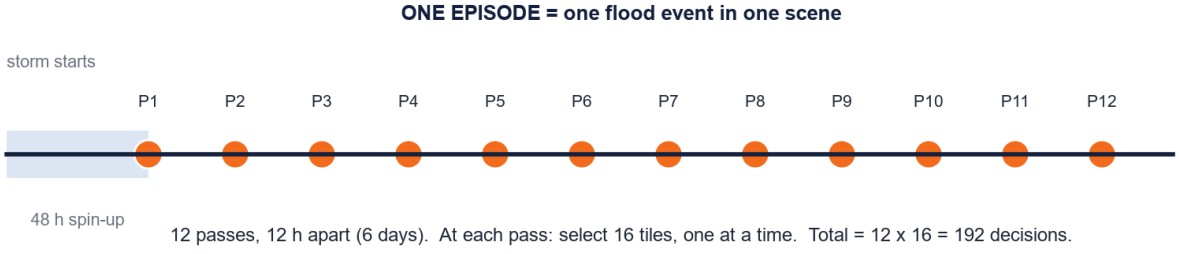


Figure 2.3 - One episode on a time line.

2.4 Why reinforcement learning?

- The decision repeats. A choice now changes what the agent knows later.
- The best choice depends on many inputs: map age, terrain, rain, forecasts and data quality.
- A simulator can make many training floods. The agent can learn by trial and error with no risk.
- A hand-made rule cannot easily balance all inputs. RL can learn the balance from the reward.

RESEARCH QUESTION

Does a forecast-aware recurrent RL agent make more accurate flood maps than hand-made tasking rules, with the same satellite limits?

Level 3. The data

3.1 The data sources

The project uses four real data sources and makes two synthetic data sets. The table gives the status of each one. **OBS** means a real observation. **REANALYSIS** means a model reconstruction after the event. **SYNTH** means data that the project makes.

#	Source	Status	What it gives	Use in the project
D1	Sen1Floods11 v1.1 (hand-labelled)	OBS	446 SAR chips (512 x 512 px, 10 m) with water labels; 11 flood events	Train and test the U-Net; calibrate
D2	Copernicus DEM GLO-30	OBS	Terrain height, 30 m (resampled to 60 m)	HAND, rivers, reach zones
D3	ERA5 daily rain (Open-Meteo)	REANALYSIS	Daily rain at each event	Storm size range; rain in replays
D4	Sentinel-1 RTC (Planetary Computer)	OBS	Assam 2016 (11 dates), Assam 2022 (25 dates)	Real-event replay test
D5	Simulated floods	SYNTH	Hourly flood maps	Training and controlled tests
D6	Synthetic forecasts and gauges	SYNTH	Rain forecasts with errors	Forecast inputs; robustness tests

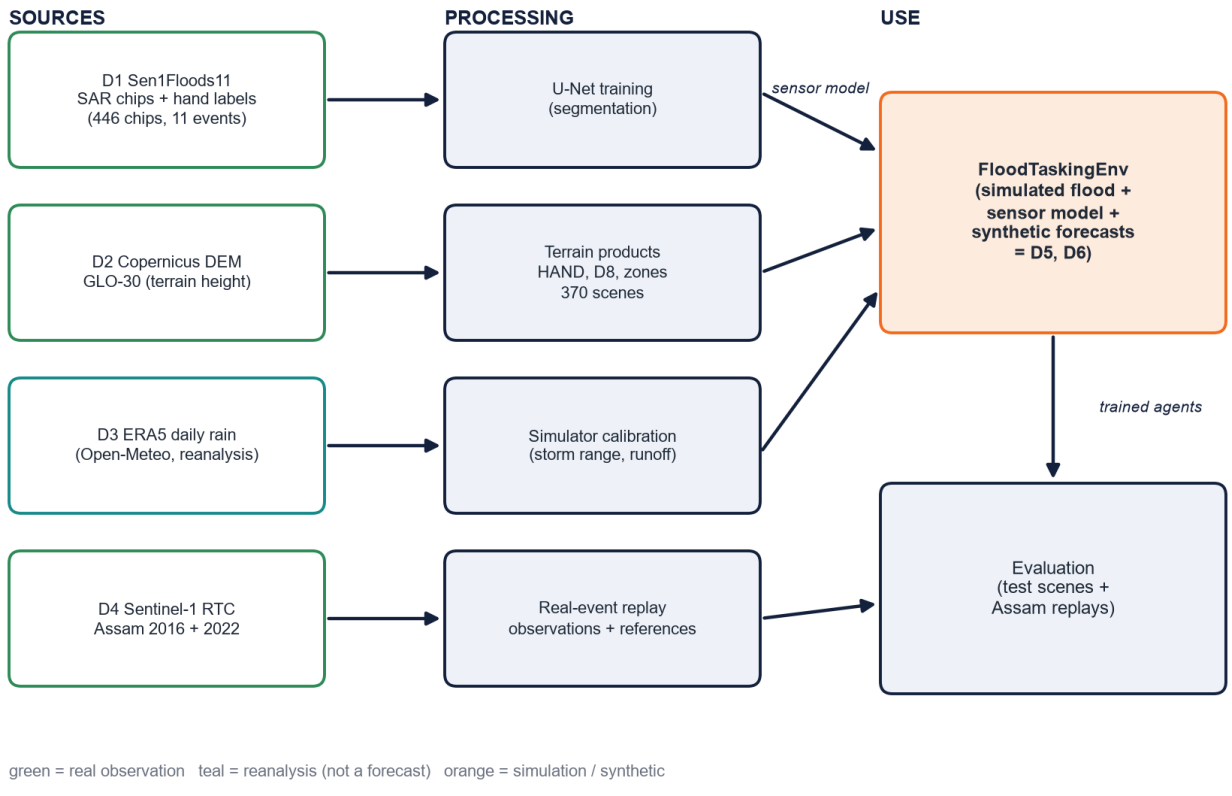


Figure 3.1 - From data sources to the environment and the evaluation.

3.2 The event split

The project splits the data by **event**, not by image. One flood event is in only one group. This rule prevents **data leakage**. Leakage occurs when test information gets into training. Then the results look better than they are.

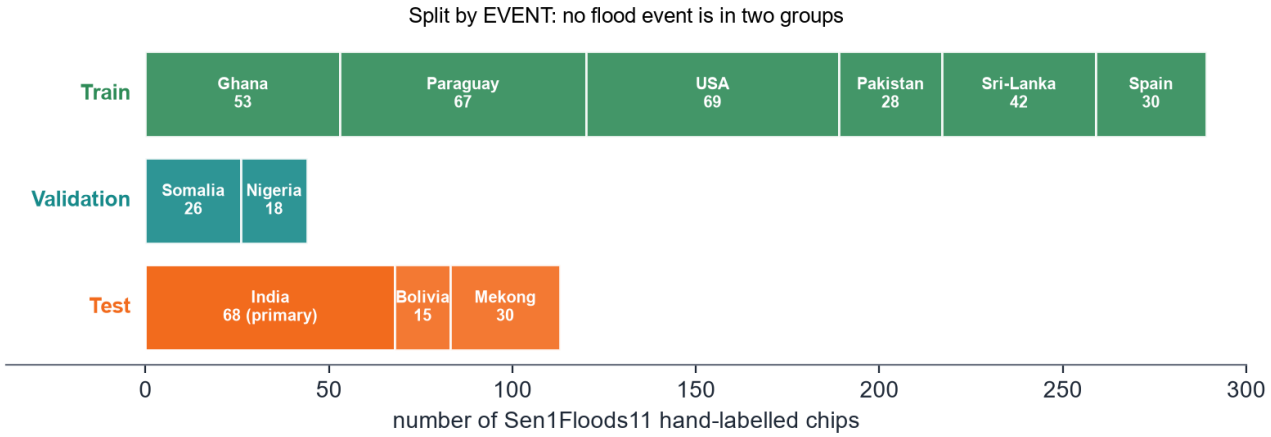


Figure 3.2 - Train, validation and test events. India (Assam) is the primary test event.

- **Train**: fit the models.
- **Validation**: select settings and checkpoints. Validation data is not used to fit the parameters.
- **Test**: measure the final result one time, after all choices are frozen. India is never used for training or tuning.

3.3 Terrain: HAND

HAND means Height Above Nearest Drainage. For each cell, HAND is the height of the cell above the river cell that its water flows to. Low HAND means the cell floods easily. The project computes HAND from the DEM in four steps (file floodtask/hydro/hand.py):

1. Fill the small holes in the DEM (priority-flood method).
2. Find the flow direction of each cell: the steepest of the 8 neighbours (D8 method).
3. Count the cells upstream of each cell (flow accumulation). Cells with 1,000 or more upstream cells are drainage (river) cells.
4. Follow the flow to the first drainage cell. HAND = cell height - drainage cell height.

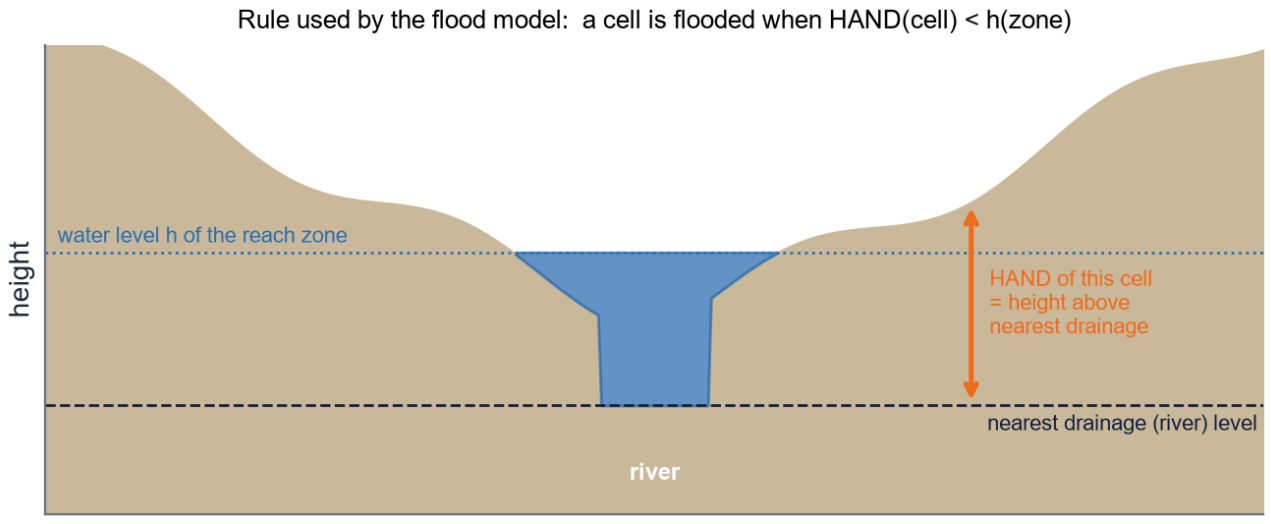


Figure 3.3 - HAND. A cell floods when its HAND is less than the water level of its reach zone.

The project groups the cells into **reach zones**. A reach zone is a part of the river and the land that drains to it. Each zone has one water level. Zones connect from upstream to downstream.

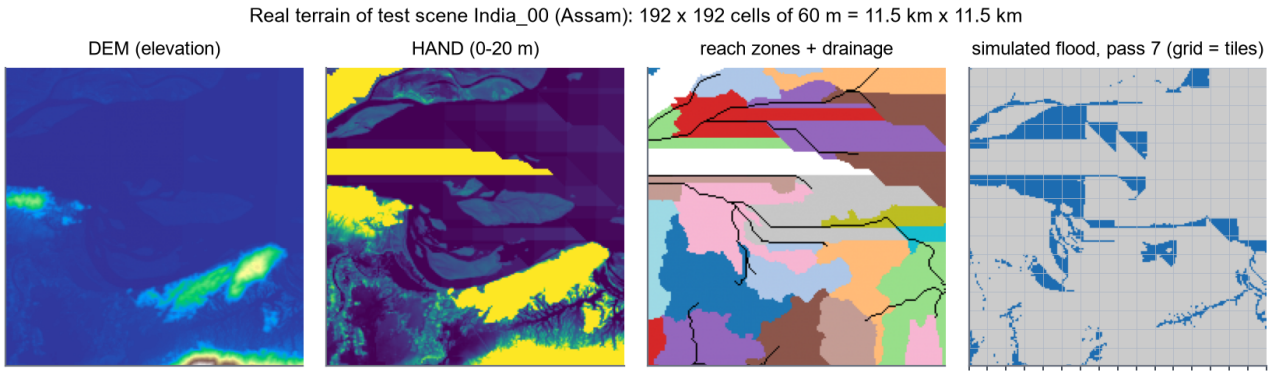


Figure 3.4 - Real data of test scene India_00 in Assam, with one simulated flood state.

The project makes **370 scenes** from DEM windows around the Sen1Floods11 chips. India has 59 scenes. The HAND terrain prior also helps the map: a logistic model gives P(water | HAND) from the training events.

$$\operatorname{logit} P(\text{water} \mid \text{HAND}) = -1.338 - 0.731 \ln(1 + \text{HAND})$$

- At HAND = 0 m the prior is 0.21. At HAND = 20 m it is only 0.03.
- The map estimator mixes this prior with each satellite observation (Bayesian fusion).

Level 4. The flood simulator and the sensor model

4.1 Why a simulator?

Sen1Floods11 has only one date for each event. RL needs floods that change with time. No open data set has hand-labelled flood maps every 12 hours. So the project simulates the flood movement on the real terrain.

> **CAUTION**
> The simulator is a conceptual model. It is not a hydrodynamic model. It has no water momentum, no levee breach and no dam operation. Level 12 lists all limits.

4.2 How the simulator works

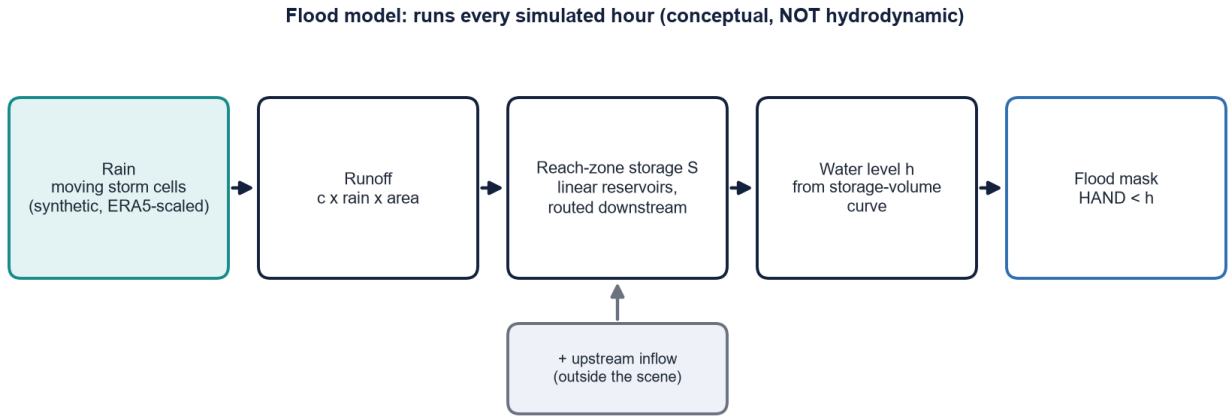


Figure 4.1 - The flood model. It runs one step for each simulated hour.

1. **Rain.** One to four Gaussian storm cells move over the area. The total rain is 27-216 mm (scaled to ERA5 events).
2. **Runoff.** A part of the rain (the runoff coefficient c) goes into the river zones.
3. **Storage.** Each reach zone is a linear reservoir. Water flows from upstream zones to downstream zones.
4. **Upstream inflow.** Rain outside the scene comes in at the inlet zone, after a delay of 12-60 h.
5. **Water level.** A storage-volume curve changes the storage S into a water level h.
6. **Flood mask.** A cell is flooded when HAND(cell) < h(zone).

$$S_z(t+1) = \left[(I-U)^{-1}(S(t)+q(t))\right]_z\left(1-\frac{1}{r_z}\right)$$

– S is the water storage of each zone. q is the inflow from rain and from upstream.

– U moves water downstream within the hour. r_z is the residence time (12-72 h).

$$F_t(x) = \mathbb{1}\left[\,\mathrm{HAND}(x) < h_{z(x)}(t)\,\right]$$

– $F_t(x) = 1$ means cell x is flooded at time t. The function 1[ ] gives 1 when the condition is true.

Each episode samples new physical settings. Thus the agent sees many different floods. The settings were **calibrated** on training events only, so that the simulated flooded fraction matches the real one (Kolmogorov-Smirnov statistic 0.13).

4.3 The sensor model

The satellite image does not show the true flood. The U-Net reads the SAR image and makes errors. The sensor model copies these errors. It gives a water probability for each cell of an imaged tile:

$$\hat{\pi} = \sigma(a\,(2\,G_b * F - 1) + \beta + \sigma_n\,\varepsilon_{corr})$$

- F is the true flood. G_b * F is a blurred copy (edges are uncertain).
- a = 3 is the signal strength. β = -1.0 is the bias. σ_n = 2.0 is the noise level.
- ε_{corr} is spatially correlated noise. σ( ) is the sigmoid function.
- The values were fitted to U-Net errors on the validation events (file results/sensor_model.json).

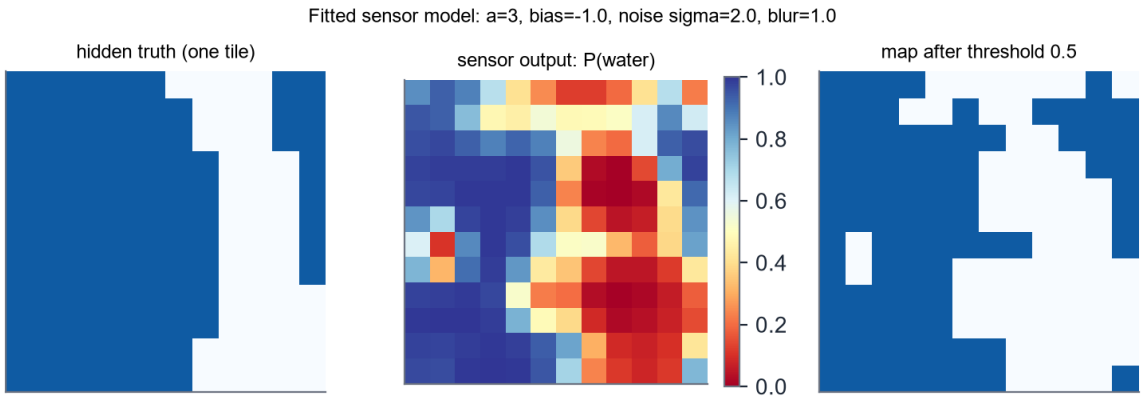


Figure 4.2 - One tile: the hidden truth, the sensor output and the map after the 0.5 threshold.

NOTE

The sensor model is pessimistic. In simulation, a single image gets IoU 0.57. The real U-Net gets about 0.70 at 60 m. So the simulated map scores are probably lower than real scores.

Level 5. The RL environment

The environment is the class FloodTaskingEnv (file floodtask/env/tasking_env.py). It joins the simulator, the sensor model, the forecasts and the map estimator. It gives the agent the observation, the mask and the reward.

5.1 Step, pass and episode

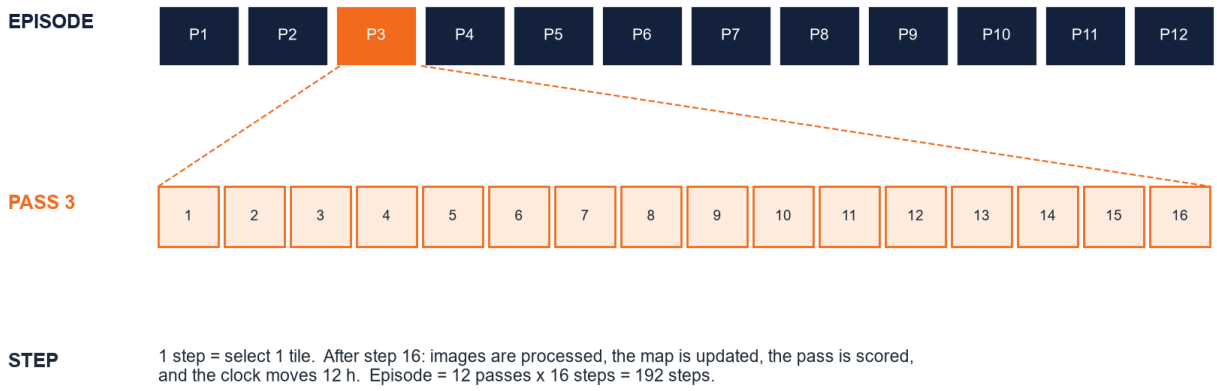


Figure 5.1 - One step selects one tile. A pass has 16 steps. An episode has 12 passes.

The agent selects one tile at each step, not 16 tiles at one time. This design makes the action space small (256 actions, not the number of 16-tile sets, which is more than 10^{25}).

5.2 The observation

The observation has three parts. It contains only information that is available at decision time. A unit test (test_no_future_truth_leakage) changes all hidden future data and checks that the observation does not change.

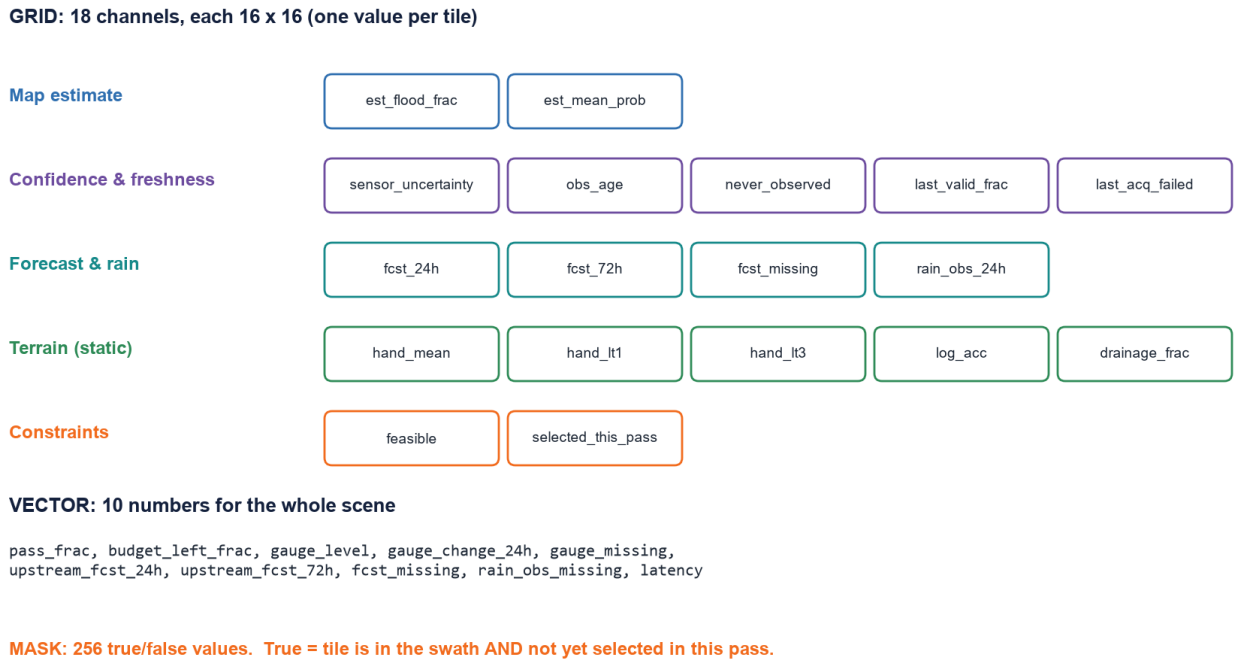


Figure 5.2 - The observation: 18 grid channels, 10 vector values and the action mask.

Channel group	Meaning
Map estimate	Fraction of the tile mapped as flood, and the mean water probability
Confidence and freshness	Sensor uncertainty (entropy), age of the last image, never-observed flag, valid fraction, last failure
Forecast and rain	Rain forecast for 24 h and 72 h (synthetic, with errors), missing flag, observed rain in the past 24 h
Terrain	Mean HAND, fraction with HAND < 1 m and < 3 m, flow accumulation, drainage fraction
Constraints	Tile is in the swath; tile is already selected in this pass

The project keeps two ideas apart. **Uncertainty** is how sure the segmentation was. **Staleness** is how old the information is. A tile can be sure and old at the same time. A combined value u is used by one baseline:

$$u_i = 1 - (1 - H_i)\, e^{-\mathrm{age}_i/\tau}$$

- H is the binary entropy of the last observation. τ is a time scale (48 h). Never-observed tiles have u = 1.

5.3 The action and the mask

The action is a tile number from 0 to 255. Some actions are not allowed: the tile is outside the swath, or the agent already selected it in this pass. The **mask** marks the allowed tiles. The network sets the score of a not-allowed tile to -10^9. Then its probability is zero. The environment also rejects an illegal action with an error.

5.4 The shared map estimator

All policies use the same map estimator. Thus the policies are different only in the tiles that they select. The estimator keeps the latest valid probability for each cell and fuses it with the HAND prior. A cell with probability 0.5 or more is "flooded" on the map.

5.5 The reward

The reward tells the agent how much one image improved the map. The project calls it the **difference reward**. For the selected tile, it compares the map errors before and after the new image:

$$r = \frac{1}{144} \sum_{x \in \mathrm{tile}} \left([3\,FN^-(x) + FP^-(x)] - [3\,FN^+(x) + FP^+(x)] \right)$$

- FN = missed flood cell (false negative). FP = false alarm (false positive).
- A missed flood costs 3 times more than a false alarm. The weight 3 was selected on validation data.
- The "-" sign marks the map before the image. The "+" sign marks the map after the image.
- Division by 144 (the cells in a tile) keeps the reward small.

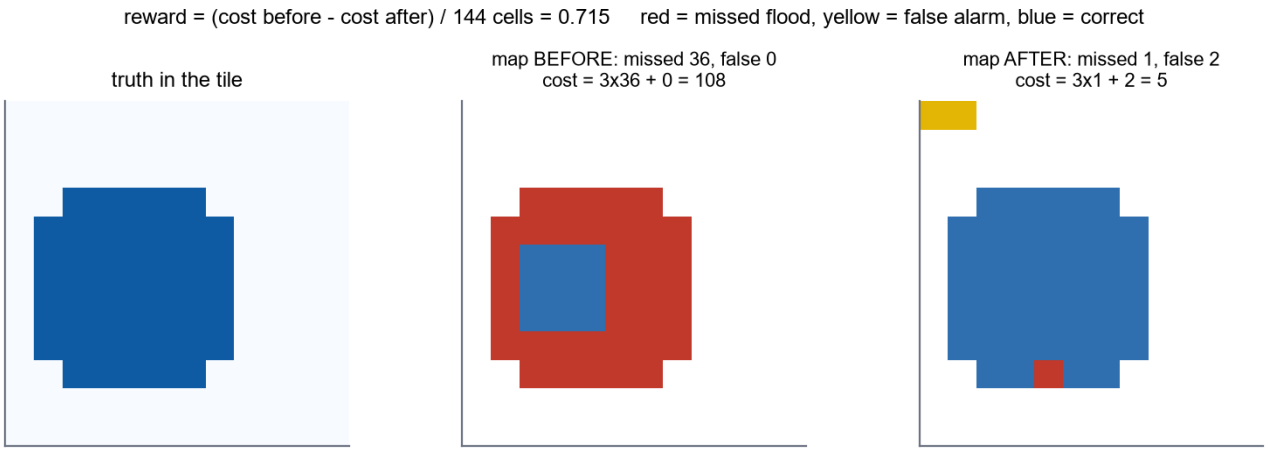


Figure 5.3 - A worked example of the difference reward for one tile.

IMPORTANT

The true flood is used only to compute the reward inside the simulator during training, and to score the maps. The agent never sees the true flood in its observation.

The reward was checked before use. The rule (requirement R12) is: the episode return must correlate with the map quality, Spearman $\rho \geq 0.7$ on validation. With weight 3, $\rho = 0.83$ for episodes and 1.00 for policies. With weight 1, ρ was only 0.77 and 0.66. An IoU-difference reward got 0.49 and was rejected.

Level 6. The neural networks

The project has two different neural networks. The U-Net reads radar images. The actor-critic makes tasking decisions.

6.1 U-Net for flood segmentation

Segmentation gives a class to each pixel. Here the classes are "water" and "not water". The U-Net has an encoder that makes the image smaller and finds features. It has a decoder that makes the image large again. Skip connections copy fine detail from the encoder to the decoder.

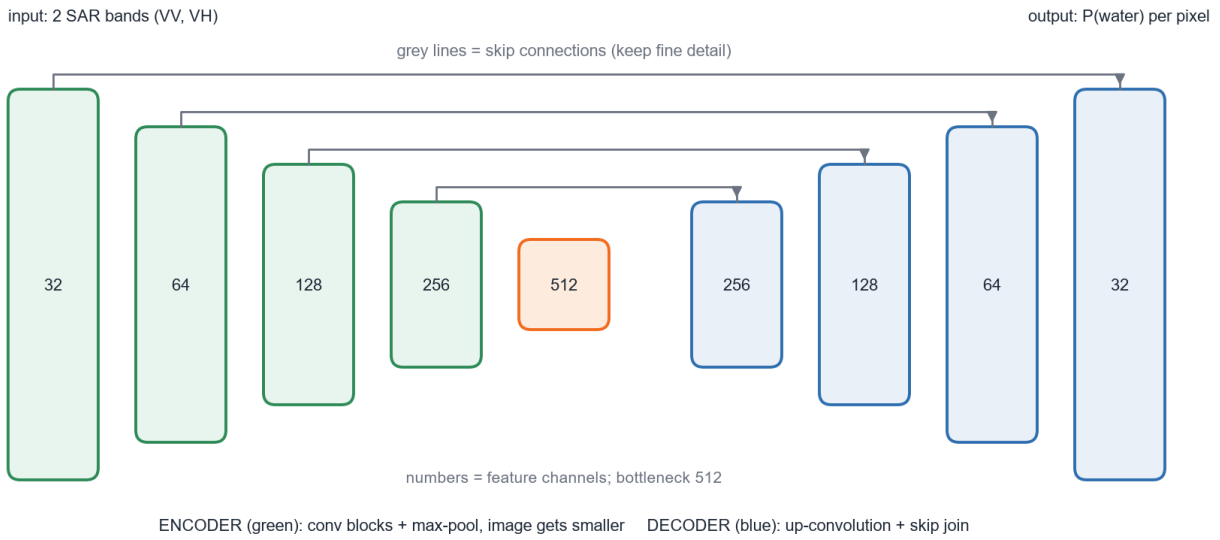


Figure 6.1 - The U-Net. Input: VV and VH radar bands. Output: water probability per pixel.

Setting	Value
Input	2 bands (VV, VH), normalised with training statistics; random 256 x 256 crops with augmentation
Depth and width	4 levels, 32 to 512 channels; dropout 0.1 in the deep layers
Loss	Weighted cross-entropy + Dice loss; label -1 (cloud or no data) is ignored
Optimizer	AdamW, learning rate 0.001, cosine schedule, 80 epochs, batch 8, mixed precision (fp16)
Calibration	Temperature scaling, fitted on validation pixels
Seeds	3 (seeds 0, 1, 2); 5.3 min per seed on the GPU

Test result: pooled IoU 0.652 (mean of 3 seeds, SD 0.036). The classic Otsu threshold gets 0.650. India is the hardest event: U-Net 0.530, Otsu 0.529. The U-Net is only equal to the classic method. This is a known limit.

6.2 The actor-critic network

The tasking agent uses one network with two outputs. The **actor** gives the policy (a probability for each tile). The **critic** gives the value V(o). The file is floodtask/rl/model.py.

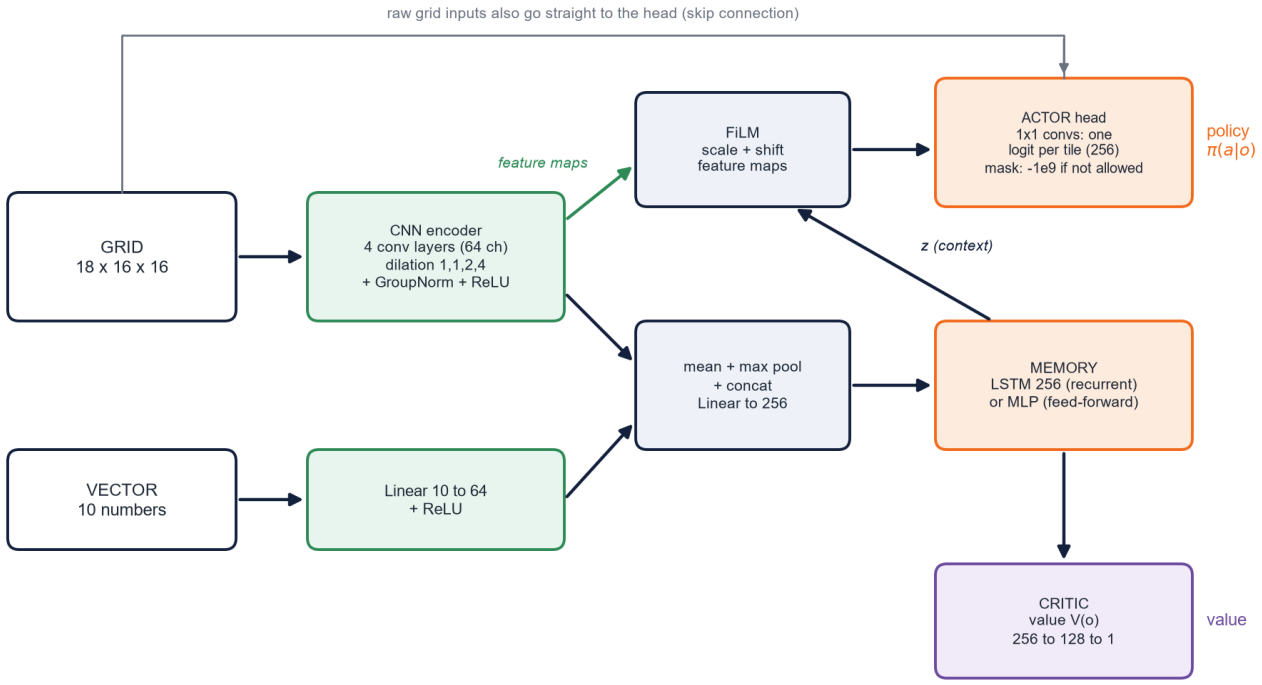


Figure 6.2 - The actor-critic. The memory block is an LSTM (recurrent) or an MLP (feed-forward).

1. The CNN encoder reads the 18 x 16 x 16 grid. Dilated convolutions see a wide area. GroupNorm keeps the signals at a good size.
2. Mean pooling and max pooling make one summary vector. A small layer reads the 10 vector values.
3. The memory block (LSTM, 256 units) keeps information from step to step. The feed-forward version has no memory.
4. **FiLM** (feature-wise linear modulation) uses the memory output z to scale and shift the feature maps.
5. The actor head gives one score (logit) for each tile. It also sees the raw inputs (skip connection).
6. The critic gives one number: the value.

$$f' = f \odot (1 + \gamma(z)) + \beta(z)$$

– f is a feature map. γ(z) and β(z) come from the memory output. The circle-dot symbol is element-wise multiplication.

6.3 From scores to an action

$$\pi(a \mid o) = \frac{e^{\ell_a}}{\sum_{b \text{ allowed}} e^{\ell_b}}, \qquad \ell_b = -10^9 \text{ if not allowed}$$

– ℓ is the score (logit) of a tile. Softmax changes scores into probabilities that add to 1.

– In training, the agent samples a tile from π. This sampling gives exploration.

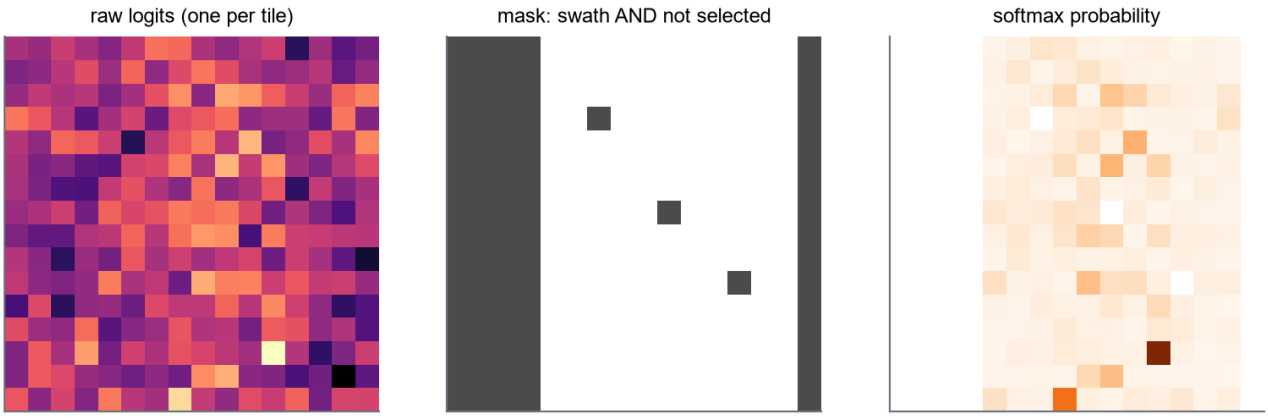


Figure 6.3 - The mask removes tiles outside the swath and tiles already selected.

LESSON FROM THE PROJECT

Without GroupNorm, the signals became 2.5 times smaller at each layer. All tile scores were almost equal and the policy could not learn. GroupNorm fixed this (docs/troubleshooting.md).

Level 7. The learning algorithm: PPO

7.1 Policy gradient: the main idea

The agent wants a large expected return J(θ). θ are the network parameters. The **policy gradient** tells how to change θ:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid o_t)\, A_t]$$

- log π(a | o) increases when the probability of action a increases.
- Multiplication by the advantage A: good actions get more probability, bad actions get less.

7.2 The advantage with GAE

The agent does not know the true advantage. It estimates it from the critic. First, it computes the **temporal-difference error** δ for each step:

$$\delta_t = r_t + \gamma V(o_{t+1}) - V(o_t)$$

- δ > 0: the step was better than the critic expected.

Generalized Advantage Estimation (GAE) adds the δ values of the next steps with a decay:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\,\delta_{t+1} + (\gamma\lambda)^2\,\delta_{t+2} + \ldots \qquad (\gamma = 0.9,\ \lambda = 0.95)$$

- λ = 0.95 controls the balance between low noise (small λ) and low bias (large λ).
- Each training episode ends after 192 steps, so the value after the last step is 0.

7.3 The clipped objective

A large policy change can destroy a good policy. PPO limits the change. It compares the new policy with the old policy by the **ratio** r:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{old}}(a_t \mid o_t)}$$

$$L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t)]$$

- ε = 0.2. The ratio can change only between 0.8 and 1.2 before the gain stops.
- The min( ) takes the more careful value. Thus the update cannot go too far.

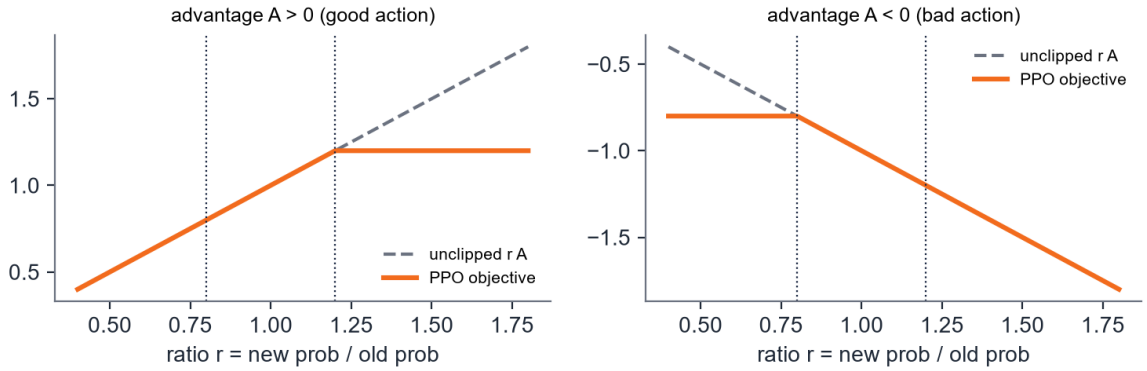


Figure 7.1 - The PPO objective is flat outside the clip range.

7.4 The full loss

$$L = -L^{CLIP} + 0.5\,(V_\theta(o_t) - \hat{G}_t)^2 - 0.003\,H[\pi_\theta(\cdot \mid o_t)]$$

- Part 1 (policy): make good actions more probable, with the clip.
- Part 2 (value, weight 0.5): make the critic predict the return.
- Part 3 (entropy, weight 0.003): keep some randomness, so the agent continues to explore.

7.5 One PPO iteration

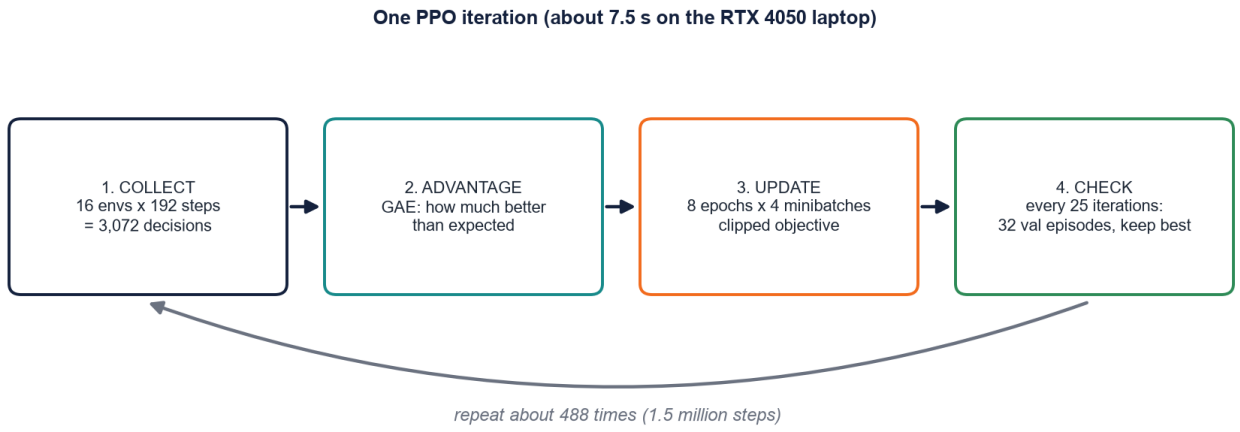


Figure 7.2 - The PPO training cycle.

1. Run 16 environments in parallel for one full episode (192 steps each). Save the observations, actions, probabilities, values and rewards.
2. Compute the advantages with GAE. Normalise them in each minibatch.
3. Do 8 epochs. In each epoch, split the 16 episodes into 4 minibatches. Do one gradient step for each minibatch.
4. Clip the gradient norm to 0.5. Decrease the learning rate linearly from 0.001 to 0.
5. Every 25 iterations, run 32 fixed validation episodes. Save the checkpoint with the best validation score.

PPO setting	Value	PPO setting	Value
Total steps	1,500,000	Discount γ	0.9
Parallel environments	16	GAE λ	0.95
Learning rate	0.001 (linear decay)	Clip ε	0.2
Epochs per iteration	8	Entropy weight	0.003
Minibatches	4	Value weight	0.5
Max gradient norm	0.5	Validation episodes	32, every 25 iterations

The settings γ, learning rate, entropy weight and epochs came from a sweep of 5 settings on validation data (400,000 steps each, feed-forward PPO). The best setting got validation IoU 0.226. All variants use the same settings.

Level 8. Baselines and training

8.1 Baselines (hand-made policies)

A **baseline** is a simple method for comparison. A good result must be better than strong baselines. Each baseline gives a score to each tile. It selects the allowed tile with the highest score (file floodtask/policies/baselines.py).

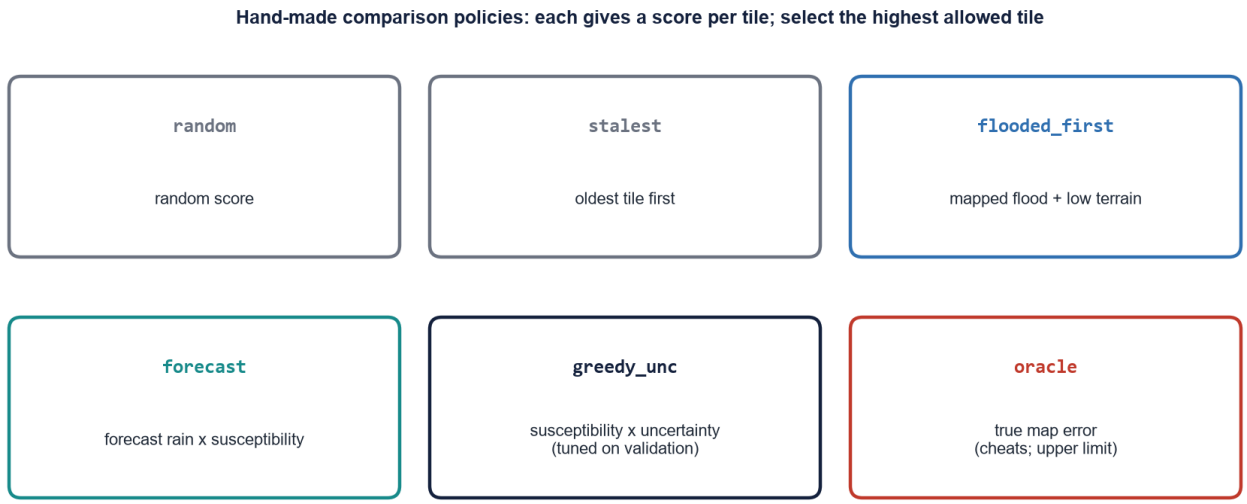


Figure 8.1 - The six comparison policies. The oracle is not a real policy: it sees the truth.

Baseline	Score of a tile
random	A random number
stalest	Age of the last image (oldest first): uniform coverage
flooded_first	Mapped flood fraction + 0.3 x low-terrain fraction (if never seen) + 0.1 x age
forecast	(forecast 24 h + 0.5 x forecast 72 h) x susceptibility + 0.2 x age
greedy_unc	susceptibility x uncertainty u; weights tuned on 48 validation episodes. The strongest baseline
oracle	The true map error of the tile. Non-operational upper reference

8.2 The five PPO variants

The project trains five variants. Each variant changes one thing. This design shows the value of each part. A study of this type is an **ablation**.

Variant	Memory	Forecast inputs	Reward	Question it answers
recurrent (main)	LSTM	yes	error reduction, FN weight 3	The proposed method
feedforward	MLP	yes	same	Does memory help?
recurrent_noforecast	LSTM	no	same	Do forecasts help?
recurrent_unc	LSTM	yes	+ 0.1 x uncertainty decrease	Does an uncertainty bonus help?
recurrent_iou	LSTM	yes	change of map IoU at pass end	Is a global reward better?



Figure 8.2 - 5 variants x 5 random seeds = 25 independent training runs.

8.3 Why five seeds?

A **random seed** fixes the random numbers of a run. RL results change much from seed to seed. Five seeds show if a result is stable. The analysis averages over the seeds and also reports the number of seeds with a positive effect.

8.4 The training runs

Each run trained for 1.5 million steps. A run took 46 to 103 minutes on the RTX 4050 laptop. All 25 runs finished. The 13 runs planned for a second machine also ran on this laptop, because the second machine failed. The settings and seeds did not change.

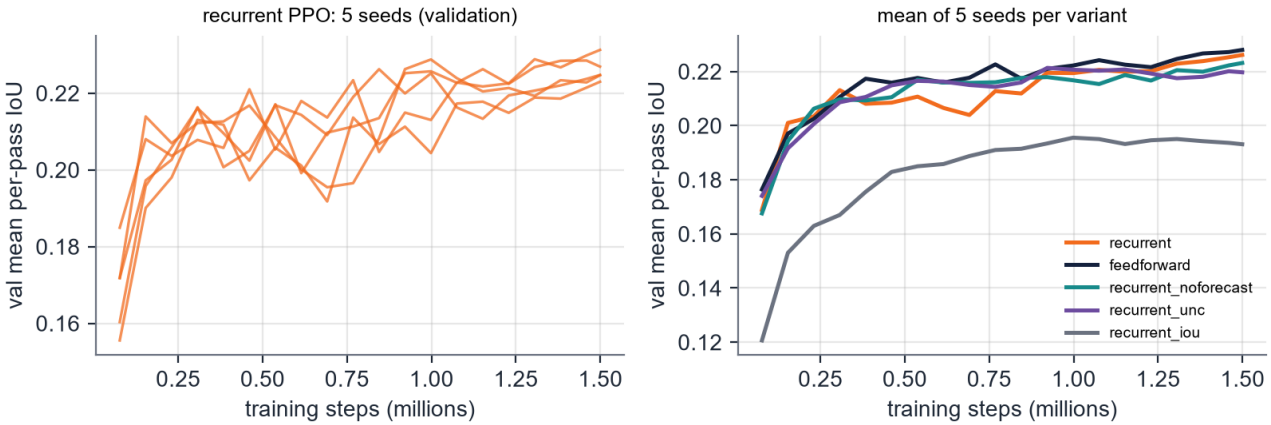


Figure 8.3 - Validation score during training (real training logs). recurrent_iou learns less.

Variant	Best validation IoU (5 seeds)	Parameters
recurrent	0.2313, 0.2288, 0.2230, 0.2285, 0.2262	769,474
feedforward	0.2323, 0.2284, 0.2322, 0.2332, 0.2309	308,930
recurrent_noforecast	0.2241, 0.2278, 0.2228, 0.2238, 0.2295	769,474
recurrent_unc	0.2200, 0.2237, 0.2302, 0.2239, 0.2247	769,474
recurrent_iou	0.1994, 0.2000, 0.1967, 0.1996, 0.1948	769,474

Level 9. Evaluation and results

9.1 The main metric: IoU

IoU (intersection over union) compares the flood map with the true flood:

$$\mathrm{IoU} = \frac{TP}{TP + FP + FN}$$

- TP = flood cells that the map shows as flood. FP = false alarms. FN = missed flood cells.
- IoU = 1 is a perfect map. IoU = 0 means no overlap.

The **primary endpoint** is the mean IoU over the passes that have a flood (true flooded fraction ≥ 0.1 %). This value rewards a map that is good early and stays good:

$$\overline{\mathrm{IoU}} = \frac{1}{|\mathcal{P}_{flood}|} \sum_{p \in \mathcal{P}_{flood}} \mathrm{IoU}_p$$

9.2 How the comparison is fair

- Every policy runs on the same **EpisodeSpecs**. A spec fixes the scene, storms, swaths, failures, sensor noise and forecast errors. Thus the comparisons are **paired**.
- The test set has 95 scenes from 3 events, with 4 episodes per scene (378 scored episodes per policy).
- The unit of analysis is the scene, not the pixel. The 95 % **confidence interval (CI)** comes from a bootstrap with 10,000 resamples.
- The test is the Wilcoxon signed-rank test. The **Holm correction** adjusts the p-values for 7 comparisons.
- The protocol was written and frozen (with a SHA-256 hash) before any test run.

9.3 Main result

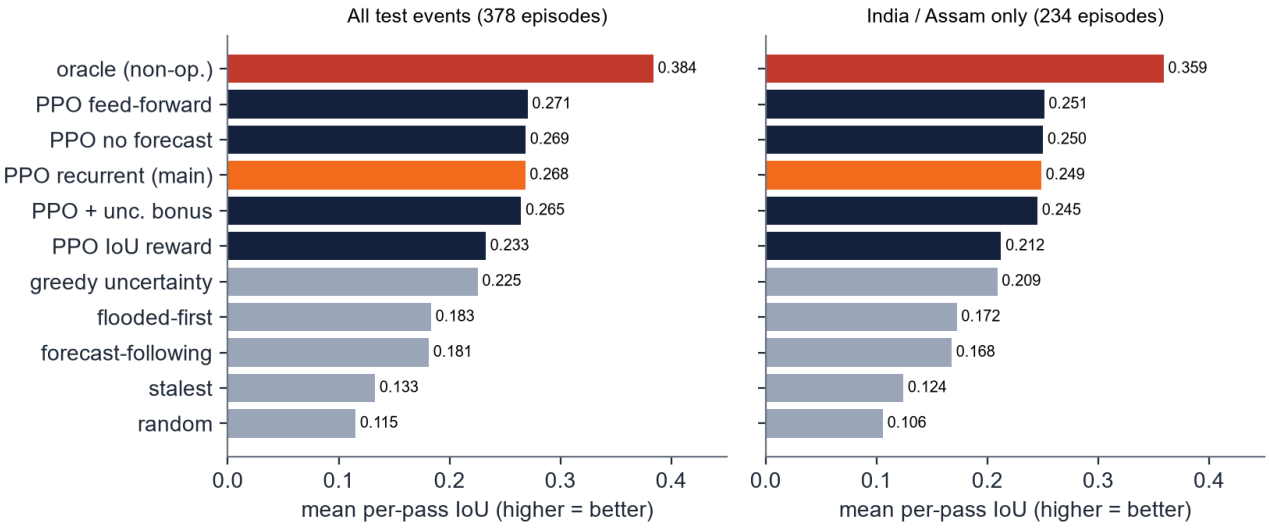


Figure 9.1 - Mean per-pass IoU on the test scenes (higher is better).

Recurrent PPO minus ...	Difference	95 % CI	Holm p	Seeds > 0	Verdict
random	+0.153	+0.144 to +0.162	1.8e-16	5/5	PPO better
stalest	+0.135	+0.127 to +0.143	1.8e-16	5/5	PPO better
flooded_first	+0.085	+0.076 to +0.094	1.8e-16	5/5	PPO better
forecast	+0.087	+0.079 to +0.095	1.8e-16	5/5	PPO better
greedy_unc (best baseline)	+0.043	+0.035 to +0.051	2.1e-14	5/5	PPO better
PPO feed-forward	-0.002	-0.004 to -0.000	0.011	1/5	memory does not help
PPO without forecast	-0.000	-0.003 to +0.002	0.40	3/5	forecasts do not help

WHAT THE RESULT MEANS

Learned tasking is clearly better than all hand-made tasking rules in simulation. The gain is +0.043 IoU over the best rule, for all 5 seeds and all 3 test events.

Memory (LSTM) and forecast inputs do not add measurable value. The feed-forward agent is slightly better.

So the evidence supports "learned tasking beats hand-built tasking in simulation". It does not support "forecast-aware memory improves tasking".

India alone (59 scenes): PPO is +0.040 better than greedy_unc (CI +0.031 to +0.049, Holm p = 1.1e-9). The direction is positive in all three events: Bolivia +0.058, India +0.040, Mekong +0.042.

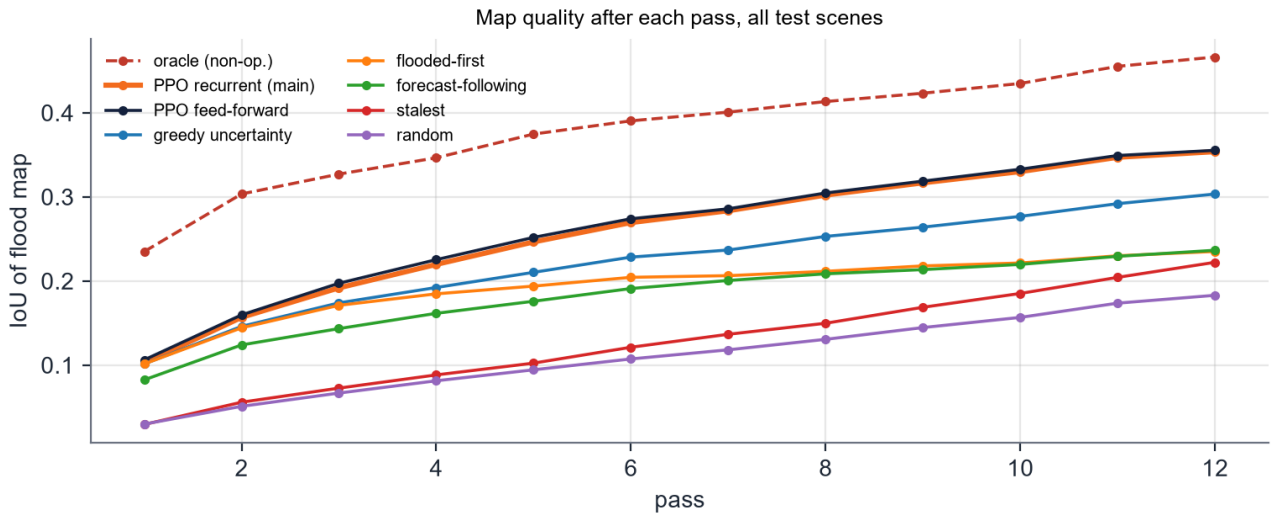


Figure 9.2 - Map IoU after each pass. PPO is above every baseline at every pass.

9.4 Robustness

The project tests the trained agents in changed conditions. The agents are not retrained.

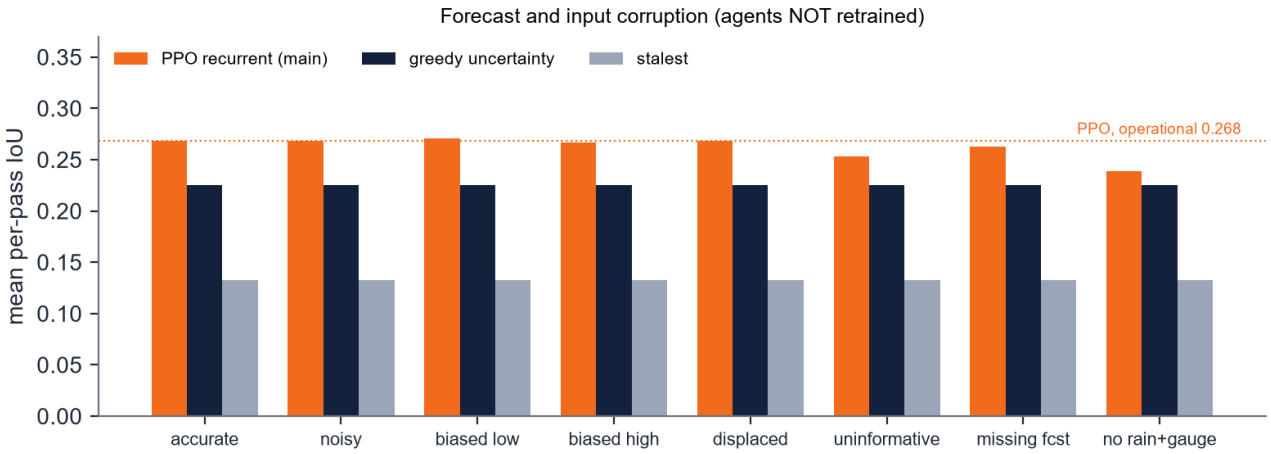


Figure 9.3 - Forecast errors change almost nothing. Missing rain and gauge data cost 11 %.

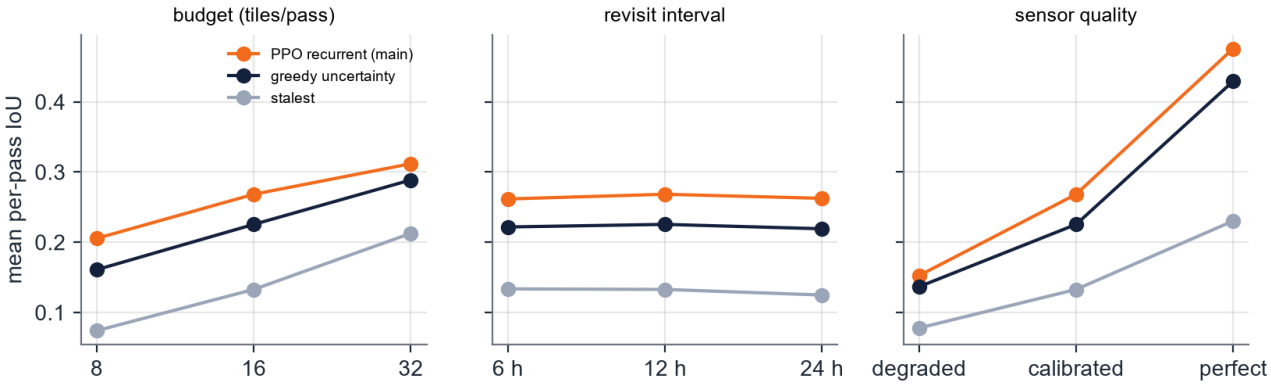


Figure 9.4 - Budget, revisit time and sensor quality. PPO stays above the baselines in all cases.

9.5 Requirements

ID	Requirement	Status
R1	Report IoU at all 12 passes	Met
R2	Final IoU ≥ best baseline; area bias ≤ 0.25	Partly met (IoU yes; pooled area bias +2.56 fails, India +0.12 passes)
R3	Not below best baseline at 8, 16, 32 tiles	Met
R4	Time to IoU 0.5 not worse	Met (weak test: medians are censored)
R5	0 illegal actions; 100 % budget	Met (6,080 episodes)
R6	Drop ≤ 10 % under every corruption	Not met (no rain and gauge: -11.0 %)
R7	Better on India; same direction on ≥ 2 of 3 events	Met (3 of 3)
R8	< 50 ms per decision, < 5 M parameters, < 2 h training	Met (5.9 ms, 0.77 M, 103 min max)
R9	One command regenerates all numbers; checksums	Met (678 files, 0 mismatches)
R10	Non-reliance conditions written	Met
R11	U-Net ≥ Otsu on test	Met (0.652 vs 0.650, small margin)
R12	Reward validity ρ ≥ 0.7 on validation	Met (0.83; on test 0.57)
R13	Hypothesis H1 (Holm $p < 0.05$, CI excludes 0)	Partly met (5 of 7 comparisons)

Level 10. Real events and the flood outlook

10.1 Real-event replay

The project tests the frozen agents on real Sentinel-1 image series over Assam. The replay environment (ReplayEnv) uses real images instead of the simulator. It has 33 non-overlapping scenes for each year.

- **Assam 2016**: 11 dates from July to September. It includes the hand-label date 2016-08-12.
- **Assam 2022**: 25 dates from May to August.
- The observation is the U-Net (seed 0) output on the real image.
- The reference map is the cells where two other U-Nets (seeds 1 + 2) and a VH Otsu threshold agree. Other cells are excluded.
- The "forecast" is ERA5 rain after the event (a pseudo-forecast). It was not available in real time.

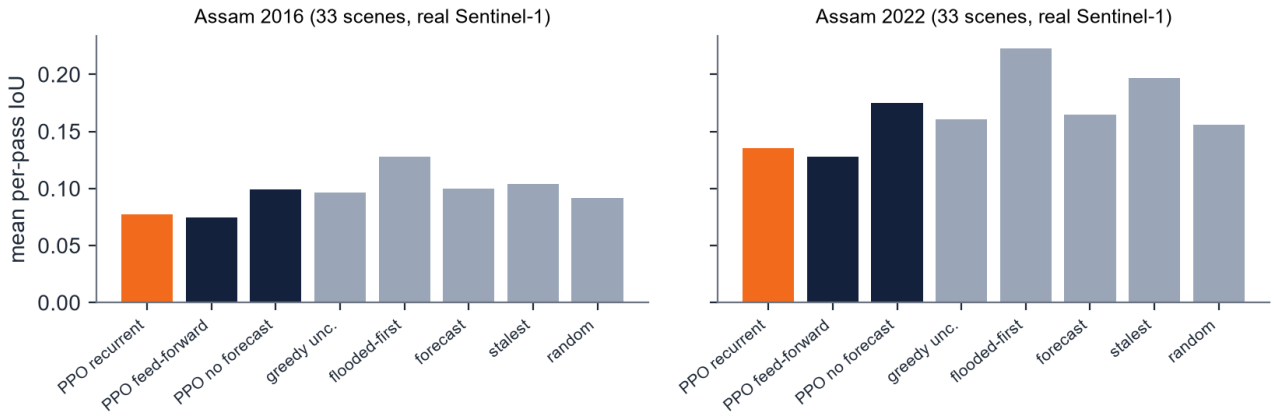


Figure 10.1 - Real Assam replays. Simple rules (flooded-first, stalest) beat PPO here.

HONEST RESULT

On the real replays, PPO does not beat the simple rules. Assam 2016: PPO 0.077, flooded-first 0.128. Assam 2022: PPO 0.135, flooded-first 0.223.

Possible causes (not tested one by one): the reference maps are noisy (reference IoU only 0.337 against hand labels), the real flood regime is different from the simulator, and the 2022 series has 25 passes, not the 12 of training.

10.2 The flood outlook

The outlook gives the probability that each 60 m cell is flooded 4, 6, 12 and 24 hours after a pass. It does not use the truth. The file is floodtask/forecast/outlook.py.

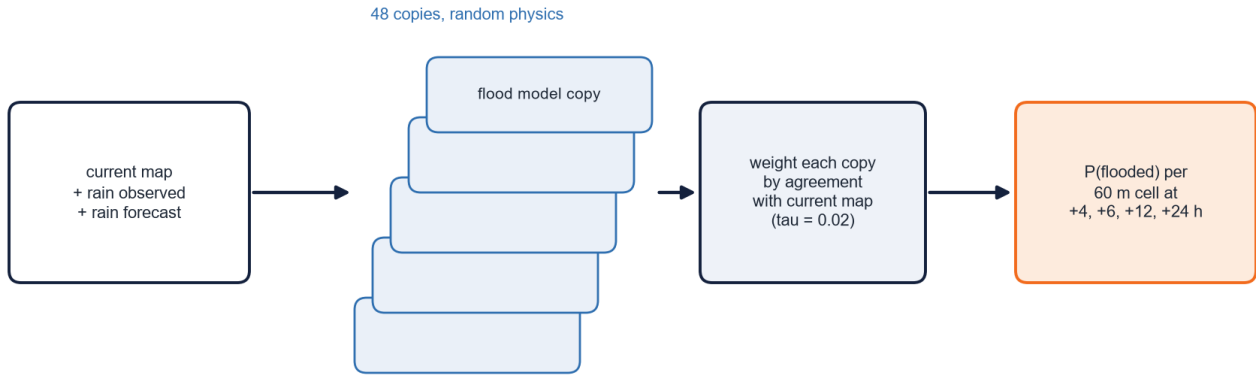


Figure 10.2 - The outlook is an ensemble of 48 flood-model copies, weighted like a particle filter.

Test	Outlook IoU	Persistence IoU
Simulated test scenes, +4 h	0.646	0.217
Simulated test scenes, +24 h	0.675	0.194
Assam 2016, next real image (median 19 days later)	0.059	0.036
Assam 2022, next real image (median 5 days later)	0.122	0.108

Persistence means "the map does not change". The simulated scores are a best case, because the same model makes the truth. On real data, the gain is small. The outlook is shown in the viewer only. No tasking policy uses it.

Level 11. The code and how to run it

11.1 Folder map

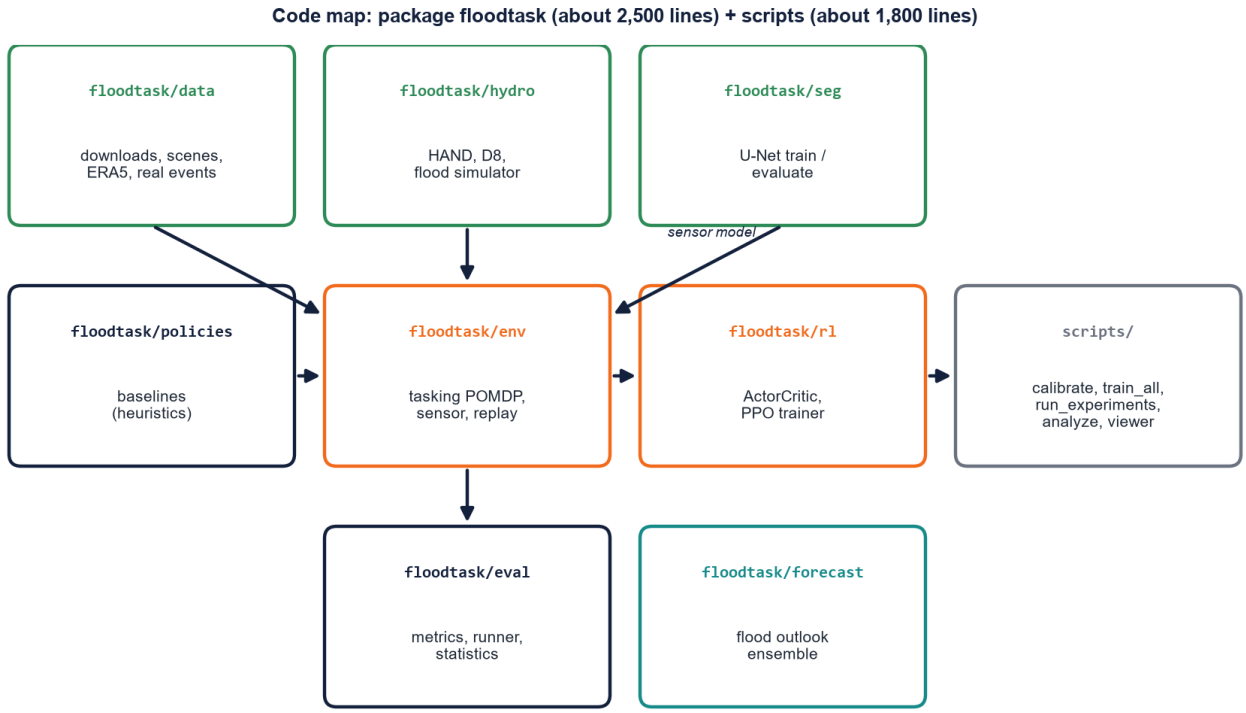


Figure 11.1 - The Python package floodtask and the scripts.

Folder or file	Contents
configs/	data.yaml (sources, split), env.yaml (environment), ppo.yaml (PPO settings), split.yaml (two-machine split)
floodtask/	The Python package (data, hydro, seg, env, policies, rl, eval, forecast)
scripts/	Calibration, tuning, training, experiments, analysis, viewer data
tests/	Unit tests: HAND, metrics, environment leakage and feasibility, reward, statistics
docs/	Requirements, architecture, data inventory, formulation, protocol, limits, acceptance report
results/	All outputs: models, logs, episodes, tables, figures, viewer

11.2 The pipeline

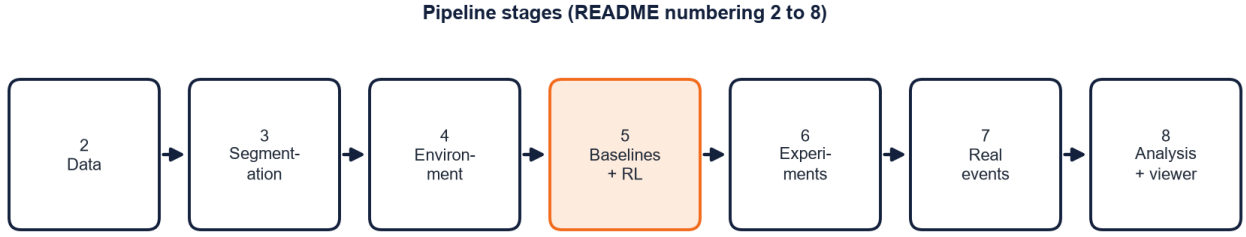


Figure 11.2 - The stages of the pipeline (numbered as in README.md).

Procedure: install the project

1. Open PowerShell in the folder C:\SEM_5_Works\RL.
2. Make a virtual environment: `python -m venv .venv`.
3. Install PyTorch 2.5.1 with CUDA 12.4.
4. Install the locked packages: `pip install -r requirements-lock.txt`.
5. Install the package: `pip install -e .`.
6. Run the tests: `python -m pytest`. Make sure that all tests pass.

Procedure: run the full pipeline

```
python -m floodtask.data.sen1floods11          # 2 download chips (+ --verify)
python -m floodtask.data.rainfall               # 2 ERA5 rain
python -m floodtask.data.scenes                 # 2 DEM, HAND, 370 scenes
python -m floodtask.seg.train --seed 0          # 3 U-Net (repeat for seeds 1, 2)
python -m floodtask.seg.evaluate --seeds 0 1 2  # 3 test the U-Net once
python scripts/fit_sensor_model.py              # 4 sensor model
python scripts/calibrate_sim.py                 # 4 simulator calibration
python scripts/tune_baselines.py                # 5 baselines (validation only)
python scripts/train_all.py                     # 5 25 PPO runs
python scripts/run_experiments.py --suite all   # 6 test suites
python scripts/run_real_event.py --event assam2016   # 7 real replay (also assam2022)
python scripts/analyze.py                       # 8 tables, statistics, figures
python scripts/make_manifest.py --verify        # 8 check all checksums
```

CAUTION

Do not use the test events to change any setting. If you change a setting after a test run, the test result is not valid any more.

11.3 The viewer app

The FloodTask viewer is one web app with five tabs: Flood maps, Results, Robustness, Real events and Training. It shows the maps of PPO and greedy_unc on the Assam replays and on simulated test scenes, with the outlook panel.

1. Run `python scripts/build_viewer_data.py`.
2. Run `python scripts/export_results_data.py`.
3. Run `python -m http.server -d results/viewer`.
4. Open http://localhost:8000 in a web browser.

11.4 An episode picture

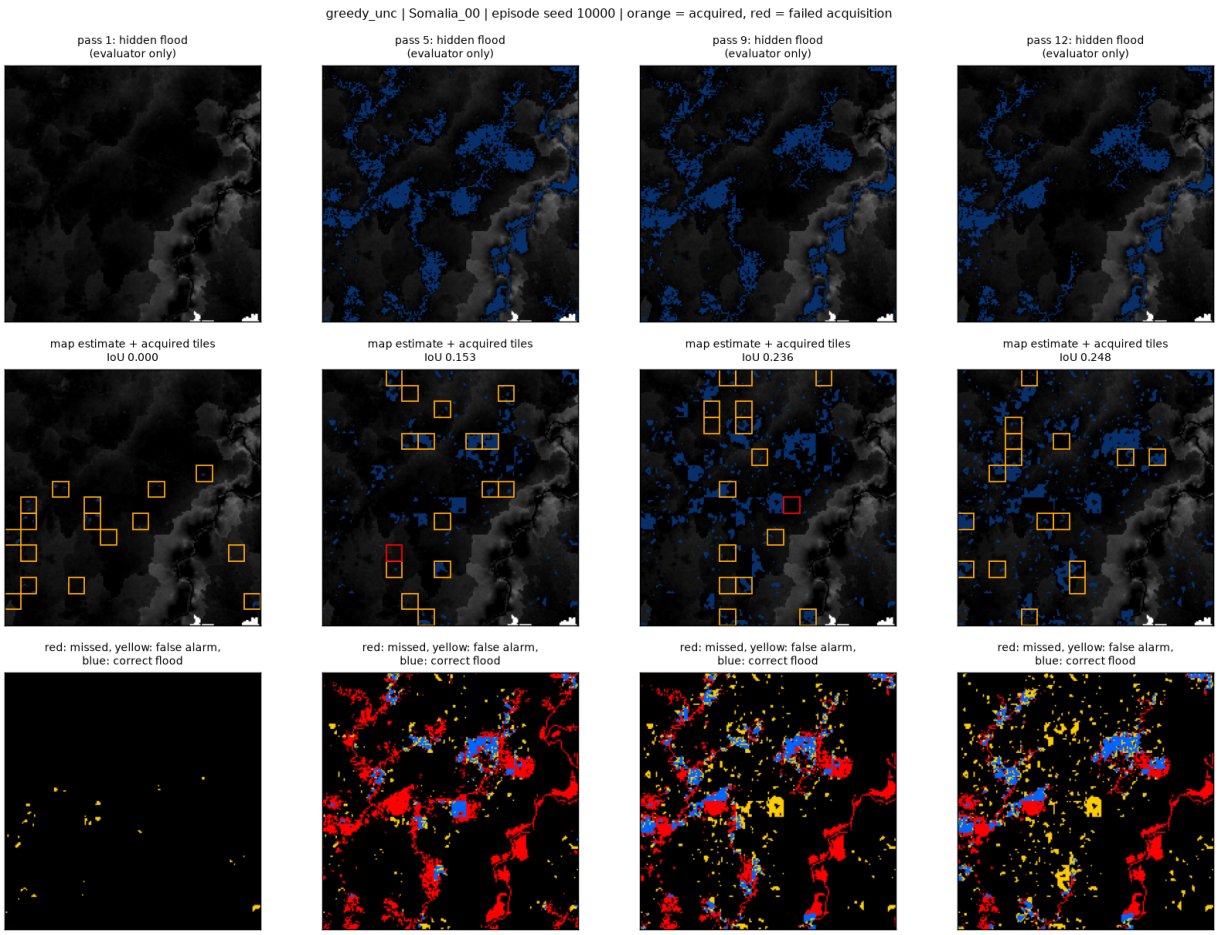


Figure 11.3 - Output of scripts/visualize_episode.py: hidden flood (top), map and tasked tiles (middle), errors (bottom).

Level 12. Limits and honest statements

A good project tells what it does not show. These are the main limits (from docs/limitations.md):

Data

- No data set has hand-labelled flood maps on many dates. The flood movement is simulated.
- No archived weather forecasts are used. Forecasts are synthetic (simulation) or ERA5 pseudo-forecasts (replay).
- No real river gauge data is used.
- The DEM is a surface model. Trees and buildings increase HAND.
- Only three test events exist, so event-level claims are qualitative.

Models

- The flood model is conceptual, not hydrodynamic.
- The sensor model uses the same noise everywhere. Real errors depend on the land cover.
- The U-Net only equals the Otsu threshold on the test events (IoU about 0.53 on India).
- The map estimator keeps the latest image. It does not move water into old tiles.

Tasking

- The tile tasking copies agile commercial SAR satellites. Sentinel-1 does not take tasking orders.
- The swath is a random band, not a real orbit.
- The agent selects one tile at a time. It does not optimise the 16 tiles together.

WARNING: DO NOT USE THE SYSTEM FOR THESE CASES

Flash floods in steep terrain, coastal storm surge or urban rain floods.

A region or sensor where the segmentation model is not validated.

Any life-safety decision without a human analyst who checks the map and its uncertainty.

The system is a research prototype. It is not an operational or certified system.

Level 13. Glossary and viva questions

13.1 Glossary

Term	Definition
Action	The decision of the agent. Here: one tile number (0-255).
Advantage	How much better an action is than the average action in that state.
Agent	The part that makes decisions. Here: the PPO network.
Baseline	A simple method for comparison.
Bootstrap	A method that resamples the data many times to get a confidence interval.
Checkpoint	A saved copy of the network parameters.
Critic	The part of the network that estimates the value.
DEM	Digital elevation model: a grid of terrain heights.
Discount (γ)	A number between 0 and 1 that makes future rewards count less.
Entropy	A measure of randomness. High entropy = the policy is not sure.
Episode	One full run: 12 passes x 16 steps = 192 steps.
EpisodeSpec	A fixed description of one episode, so that all policies face the same conditions.
ERA5	A weather reanalysis (a reconstruction after the event).
FiLM	Feature-wise linear modulation: scale and shift features with a context vector.
GAE	Generalized advantage estimation: a weighted sum of TD errors.
HAND	Height above nearest drainage.
Holm correction	A method that adjusts p-values when there are many tests.
IoU	Intersection over union = TP / (TP + FP + FN).
LSTM	Long short-term memory: a recurrent layer with a memory.
Mask	A list of the allowed actions.
MDP / POMDP	Markov decision process / partially observable MDP.
Observation	What the agent sees. It is not the full state.
Oracle	A policy that uses the truth. It is only an upper reference.
Pass	One satellite opportunity: 16 tile selections.
Policy (π)	The strategy: the probability of each action.
PPO	Proximal policy optimisation: an actor-critic algorithm with a clipped update.
Return (G)	The discounted sum of future rewards.
Reward	A number from the environment after each action.
SAR	Synthetic aperture radar. It sees through clouds.
Seed	A number that fixes the random numbers of a run.
Segmentation	A class for each pixel (water / not water).
Swath	The area that the satellite can reach at a pass.
Tile	12 x 12 cells of 60 m = 720 m square. 256 tiles in a scene.
U-Net	An encoder-decoder CNN for segmentation.
Value V(s)	The expected return from a state.

13.2 Likely viva questions

Q. What problem does the project solve?

A. It selects which 16 of 256 tiles a radar satellite images at each pass. The goal is a correct and current flood map.

Q. Why reinforcement learning and not supervised learning?

A. No data set gives the correct tile choice. Only the effect on the map is known. RL learns from this effect (the reward).

Q. Why is it a POMDP?

A. The agent sees only old and noisy images. It does not see the true flood.

Q. Why PPO?

A. PPO is stable. It works with a large discrete action set and a mask. It needs few settings.

Q. What does the critic do?

A. It estimates the value. The advantage = return - value tells which actions were good.

Q. Why one tile per step?

A. A choice of 16 tiles together has more than 10^{25} options. One tile per step has 256.

Q. How do you prevent cheating with the truth?

A. The truth is used only in the reward and the scores. A unit test changes the hidden future and checks that the observation stays the same.

Q. Why the missed-flood weight 3?

A. With weight 3, the return correlates best with the map quality on validation ($\rho = 0.83$). A missed flood is also more dangerous.

Q. What is the main result?

A. PPO beats the best hand-made rule by +0.043 IoU in simulation, for all seeds and all test events.

Q. Did memory and forecasts help?

A. No. The feed-forward agent and the agent without forecasts are equal or slightly better.

Q. Does it work on real data?

A. Not yet. On the Assam replays, simple rules are better. The reference maps are also noisy.

Q. Why split by event?

A. Images of one event are similar. A split by image leaks test information into training.

Q. What is HAND?

A. The height of a cell above the river cell that it drains to. Low HAND = high flood risk.

Q. What would you do next?

A. Train with real multi-date data, use real forecasts and gauges, model real orbits, and train on longer pass sequences.